

System Design Document: Society Maintenance Tracker

Scope: Architecture, Complaint History Model, Overdue Detection, Photo Pipeline & Notifications | **Word Count:** ~680 words

1. Architectural Overview

The **Society Maintenance Tracker** is engineered as a decoupled three-tier architecture with strict Role-Based Access Control (RBAC):

- **Presentation Tier (React 18 + Tailwind CSS + Vite):** Single-page application providing dedicated portals for Residents (ticket lodging, timeline tracking) and Admins (filtering, status transitions, SLA configuration, Recharts visual analytics).
- **Application Tier (Node.js + Express + TypeScript):** Modular REST API using a Controller-Service-Repository pattern with JWT authentication, multipart file handling, and email dispatch services.
- **Persistence Tier (Prisma ORM + SQLite / PostgreSQL):** Normalized schema enforcing referential integrity, indexing on high-frequency filters (status, category, priority, createdAt), and an append-only audit trail.

2. Complaint Status Lifecycle & History Model

The complaint state machine follows a unidirectional progression: **OPEN** → **IN_PROGRESS** → **RESOLVED (Closed)**. Rather than updating status columns in-place, every state transition is recorded as an immutable event in the ComplaintHistory table.

- **Audit Schema:** Tracks complaintId, actorId (User FK), action (CREATED, STATUS_CHANGE, PRIORITY_CHANGE), fromStatus/toStatus, fromPriority/toPriority, note (technician remarks), and createdAt.
- **Non-Repudiation & Transparency:** Both residents and committee members can inspect the full chronological timeline with timestamps and actor details.
- **SLA & Performance Metrics:** Delays between creation, vendor assignment, and resolution provide quantitative turnaround metrics.
- **Atomicity:** Prisma atomic transactions guarantee that complaint record updates and history logs are written synchronously.

3. Dynamic Overdue SLA Detection & Admin Priority Surfacing

To avoid database bloat and race conditions from cron jobs, overdue evaluation is computed dynamically at query time using a configurable society threshold (T_{days} , default: 3 days):

Formula: $Days\ Open = \text{floor}((Now - CreatedAt) / 86,400,000\ ms) \mid isOverdue = (Status \neq 'RESOLVED') \text{ AND } (Days\ Open \geq T_days)$

Admin Queue Surfacing Hierarchy: Complaints in the admin manager are ranked by a multi-tiered sorting comparator: (1) Overdue items first, (2) Active status (Open/In Progress before Resolved), (3) Priority weight (High > Medium > Low), (4) Creation date descending. Admins can adjust the threshold dynamically in Settings without server restarts.

4. Photo Attachment Handling Pipeline

1. **Upload Pipeline:** Managed via Multer storage middleware with strict MIME-type checking (JPEG, PNG, WEBP) and a 5MB size limit.
2. **Collision Prevention:** Files receive randomized unique filenames (photo-timestamp-random.ext) and are served via static Express middleware.
3. **Cloud Ready:** The storage interface abstracts file paths, allowing zero-code migration to AWS S3, Cloudinary, or Supabase Storage.

5. Notification & Email Event Flow

- **Status Transition Notifications:** Whenever an admin updates a ticket (e.g. to In Progress or Resolved), an HTML email is automatically dispatched to the resident containing status changes, priority, and admin notes.
- **Important Notice Broadcast:** When a notice is marked `isImportant: true`, a broadcast notification is emailed to all registered residents.
- **Development Fallback:** Automatically creates ephemeral test accounts on Ethereum Mail with direct web inspection links in console logs.